VM 4 Penetration Testing Report

Docker / Portainer / OctoberCMS Security Assessment

Target VM	192.168.56.101
Platform	CentOS 7 Docker Host
Main Components	Portainer, OctoberCMS, Docker, MySQL
Assessment Type	Local Virtual Machine Security Review
Prepared For	Project Delivery

This final report consolidates the analysis, evidence, remediation steps, and verification results for the VM 4 Docker-based environment. The writing has been refined for a clear and professional project submission while preserving the command evidence and screenshots from the working notes.

1. Executive Summary

The assessed virtual machine is a CentOS 7 Docker host running Portainer and OctoberCMS containers. The main security risk is the exposure of Docker management functionality through Portainer, combined with Docker socket access and weak secrets in container metadata. These issues can allow an attacker with access to the management interface or Docker host to inspect containers, retrieve credentials, and potentially control the Docker environment.

The strongest findings are the exposed Portainer management interface and exposed MySQL root credentials inside Docker metadata. Remediation focused on reducing network exposure, binding services to localhost, replacing weak credentials, and running the active OctoberCMS container as a non-root user.

The detailed findings are presented in the following sections with evidence, impact, remediation actions, and verification. The summary table was removed to keep the final submission clean and avoid cramped layout.

2. Environment Overview

The VM hosts Docker containers for Portainer and OctoberCMS. The active network evidence showed Portainer published on ports 8000 and 9000 and OctoberCMS previously published on port 80. After remediation, the active services were bound to 127.0.0.1 to prevent direct access from the VM network.

```
docker ps -a
nmap -sV -p 8000,9000 192.168.56.101
Test-NetConnection 192.168.56.101 -Port 9000
```

Evidence - Docker containers and exposed service scan

```
[root@localhost ~]# docker ps -a
CONTAINER ID        IMAGE                  COMMAND                  CREATED             STATUS                          P
ORTS                                       NAMES
29103d6a205b        octobercms:1.0.412     "bash -c /usr/local/…"   6 years ago         Restarting (1) 13 seconds ago
                                           brave_napier
fec06933ad6c        octobercms:1.0.412     "bash -c /usr/local/…"   6 years ago         Exited (137) 6 years ago
                                           relaxed_swartz
57a12ba84a3f        octobercms:1.0.412     "bash -c /usr/local/…"   6 years ago         Exited (137) 6 years ago
                                           zealous_euclid
84c41ebf913f        octobercms:1.0.412     "bash -c /usr/local/…"   6 years ago         Exited (255) 6 years ago        0
.0.0.0:80->80/tcp                          sad_yonath
08bde6e6e517        portainer/portainer    "/portainer"             6 years ago         Up About an hour                0
.0.0.0:8000->8000/tcp, 0.0.0.0:9000->9000/tcp   portainer
```

```
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-28 23:38 +0200
Nmap scan report for 192.168.56.101
Host is up (0.0010s latency).

PORT     STATE SERVICE     VERSION
8000/tcp open  nagios-nsca Nagios NSCA
|_http-title: Site doesn't have a title (text/plain; charset=utf-8).
9000/tcp open  http        Golang net/http server
| fingerprint-strings:
|   GenericLines:
|     HTTP/1.1 400 Bad Request
|     Content-Type: text/plain; charset=utf-8
|     Connection: close
|     Request
|   GetRequest, HTTPOptions:
|     HTTP/1.0 200 OK
|     Accept-Ranges: bytes
|     Cache-Control: max-age=31536000
|     Content-Length: 23032
|     Content-Type: text/html; charset=utf-8
|     Last-Modified: Wed, 04 Dec 2019 04:22:15 GMT
|     X-Content-Type-Options: nosniff
|     X-Xss-Protection: 1; mode=block
|     Date: Thu, 28 May 2026 21:39:01 GMT
|     <!DOCTYPE html><html lang="en" ng-app="portainer">
|     <head>
|     <meta charset="utf-8">
```

3. Finding 1 - Exposed Portainer Docker Management Interface

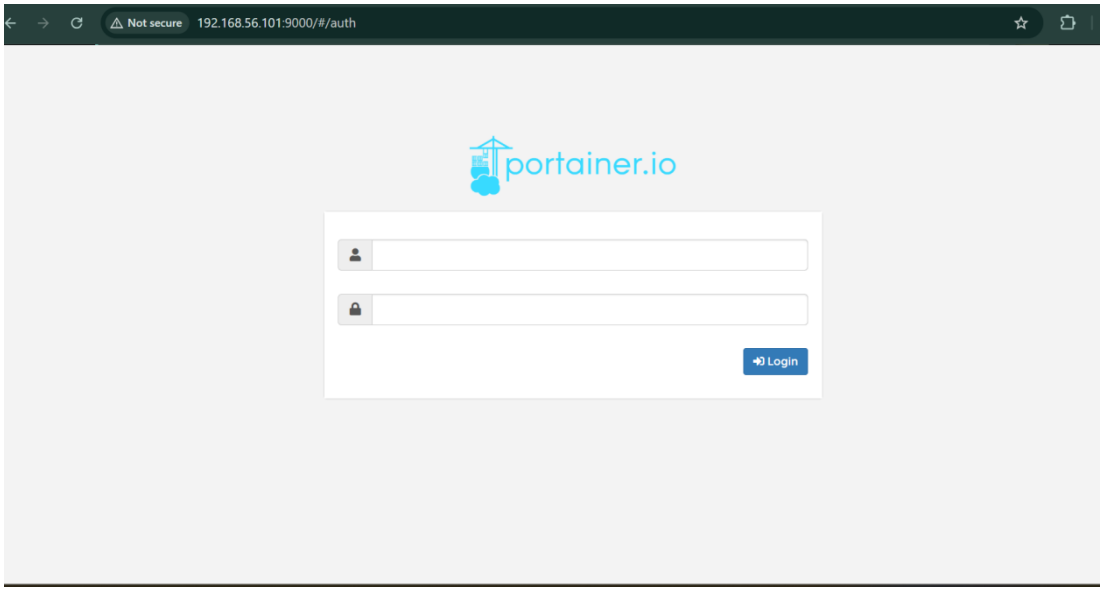
Severity: Critical

Portainer was exposed through ports 8000 and 9000. Portainer is a Docker management interface, not a normal web application. If an unauthorized user gains access, they may manage containers, images, volumes, and Docker settings.

Evidence points:

- Portainer service was exposed on ports 8000 and 9000.
- The Portainer login page was accessible from the browser using the VM IP address.
- The Portainer container was running with published ports on all interfaces.
- External connectivity testing confirmed that port 9000 was reachable before remediation.

Evidence - Portainer login page and published ports




```
[root@localhost ~]# docker ps
CONTAINER ID        IMAGE                  COMMAND                  CREATED             STATUS                          PORTS
                    NAMES
29103d6a205b        octobercms:1.0.412     "bash -c /usr/local/…"   6 years ago         Restarting (1) 47 seconds ago
                    brave_napier
08bde6e6e517        portainer/portainer    "/portainer"             6 years ago         Up 2 hours                      0.0.0.0:8000->
8000/tcp, 0.0.0.0:9000->9000/tcp   portainer
[root@localhost ~]# ss -tulnp | grep -E "8000|9000"
tcp    LISTEN     0      128      :::9000                 :::*                   users:(("docker-proxy",pid=2402,fd=4))
tcp    LISTEN     0      128      :::8000                 :::*                   users:(("docker-proxy",pid=2414,fd=4))
```

Why this is a vulnerability

An exposed Docker management interface can lead to unauthorized administration of the container environment. If credentials are weak, reused, or bypassed, the attacker may inspect container configuration, retrieve secrets, stop services, or deploy malicious containers.

Remediation

External access to Portainer was restricted. The stronger fix was to recreate Portainer so it listens only on localhost. This allows administration through SSH tunneling while preventing direct access from the VM network.

```
iptables -I DOCKER-USER -i enp0s17 -p tcp --dport 8000 -j DROP
iptables -I DOCKER-USER -i enp0s17 -p tcp --dport 9000 -j DROP
iptables-save > /etc/sysconfig/iptables
```

```
docker stop portainer
docker rm portainer
docker run -d -p 127.0.0.1:8000:8000 -p 127.0.0.1:9000:9000 --name portainer --restart=always -v \
  /var/run/docker.sock:/var/run/docker.sock -v portainer_data:/data portainer/portainer
```

Verification

```
Test-NetConnection 192.168.56.101 -Port 9000
# Expected result after remediation: TcpTestSucceeded : False
```

Evidence - firewall/localhost restriction and failed external connection after remediation

```
[root@localhost ~]# iptables-save > /etc/sysconfig/iptables
[root@localhost ~]# cat /etc/sysconfig/iptables
# Generated by iptables-save v1.4.21 on Thu May 28 23:57:39 2026
*filter
:INPUT ACCEPT [10:672]
:FORWARD DROP [0:0]
:OUTPUT ACCEPT [7:2428]
:DOCKER - [0:0]
:DOCKER-ISOLATION-STAGE-1 - [0:0]
:DOCKER-ISOLATION-STAGE-2 - [0:0]
:DOCKER-USER - [0:0]
-A INPUT -p tcp -m tcp --dport 8000 -j DROP
-A INPUT -p tcp -m tcp --dport 9000 -j DROP
-A FORWARD -j DOCKER-USER
-A FORWARD -j DOCKER-ISOLATION-STAGE-1
-A FORWARD -o docker0 -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
-A FORWARD -o docker0 -j DOCKER
-A FORWARD -i docker0 ! -o docker0 -j ACCEPT
-A FORWARD -i docker0 -o docker0 -j ACCEPT
-A DOCKER -d 172.17.0.3/32 ! -i docker0 -o docker0 -p tcp -m tcp --dport 9000 -j ACCEPT
-A DOCKER -d 172.17.0.3/32 ! -i docker0 -o docker0 -p tcp -m tcp --dport 8000 -j ACCEPT
-A DOCKER-ISOLATION-STAGE-1 -i docker0 ! -o docker0 -j DOCKER-ISOLATION-STAGE-2
-A DOCKER-ISOLATION-STAGE-1 -j RETURN
-A DOCKER-ISOLATION-STAGE-2 -o docker0 -j DROP
-A DOCKER-ISOLATION-STAGE-2 -j RETURN
-A DOCKER-USER -j RETURN
COMMIT
# Completed on Thu May 28 23:57:39 2026
```

```
PS C:\Users\DELL> Test-NetConnection 192.168.56.101 -Port 9000


ComputerName     : 192.168.56.101
RemoteAddress    : 192.168.56.101
RemotePort       : 9000
InterfaceAlias   : Ethernet 5
SourceAddress    : 192.168.56.1
TcpTestSucceeded : True
```

```
PS C:\Users\DELL> Test-NetConnection 192.168.56.101 -Port 9000
WARNING: TCP connect to (192.168.56.101 : 9000) failed


ComputerName            : 192.168.56.101
RemoteAddress           : 192.168.56.101
RemotePort              : 9000
InterfaceAlias          : Ethernet 5
SourceAddress           : 192.168.56.1
PingSucceeded           : True
PingReplyDetails (RTT) : 0 ms
TcpTestSucceeded        : False
```

4. Finding 2 - Docker Socket Exposure to Portainer

Severity: High / Critical

Portainer had access to the host Docker socket, /var/run/docker.sock. This socket allows communication with the Docker daemon and is commonly used by Portainer to manage local containers.

```
docker inspect portainer | grep -i "/var/run/docker.sock"
docker inspect portainer | grep -A 10 '"Binds"'
docker inspect portainer | grep -i "HostConfig" -A 80
```

Evidence - Docker socket mounted inside Portainer

Why this is a vulnerability

The Docker socket provides powerful control over the Docker daemon. If an attacker gains access to Portainer while this socket is mounted, they may control containers, inspect environment variables, access volumes, or create containers that interact with host resources.

Remediation

- Do not expose Portainer directly to the network.
- Bind Portainer to 127.0.0.1 and use SSH tunneling or a VPN for administration.
- Restrict Portainer accounts to trusted administrators only.
- Review whether direct Docker socket access is required. If possible, use a safer remote endpoint or a properly protected Portainer agent setup.
- Monitor Portainer and Docker activity logs for unexpected container creation or configuration changes.

```
ssh -L 9000:127.0.0.1:9000 root@192.168.56.101
# Then browse to: http://127.0.0.1:9000
```

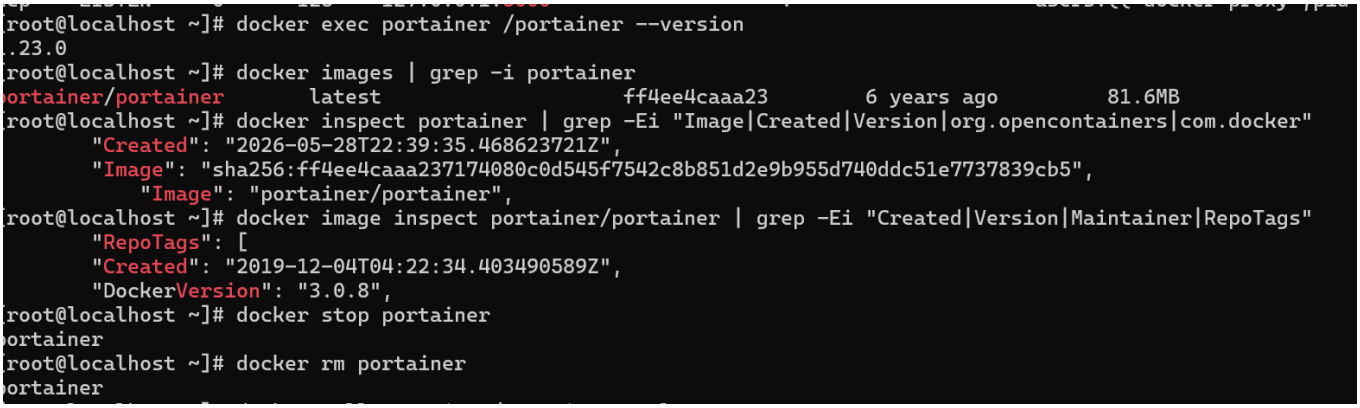
5. Finding 3 - Outdated Portainer Image

Severity: High

The Portainer image in use was portainer/portainer:latest, created in 2019 and reported by Docker as several years old. Older management interfaces may contain known vulnerabilities, missing security fixes, or unsupported components.

```
docker images | grep -i portainer
docker inspect portainer | grep -Ei "Image|Created|Version|org.opencontainers|com.docker"
docker image inspect portainer/portainer | grep -Ei "Created|Version|Maintainer|RepoTags"
```

Evidence - outdated Portainer image details



Why this is a vulnerability

Portainer manages Docker resources. Running an outdated version increases the chance that known weaknesses or missing security controls can affect the Docker environment.

Remediation

```
docker stop portainer
docker rm portainer
docker pull portainer/portainer-ce:latest
docker run -d -p 127.0.0.1:8000:8000 -p 127.0.0.1:9000:9000 --name portainer --restart=always -v \
  /var/run/docker.sock:/var/run/docker.sock -v portainer_data:/data portainer/portainer-ce:latest
```

During remediation, pulling the latest image failed because the VM had DNS/certificate trust issues. Until the update can be completed, the temporary mitigation is to keep Portainer bound to 127.0.0.1 and not expose it directly to the VM network.

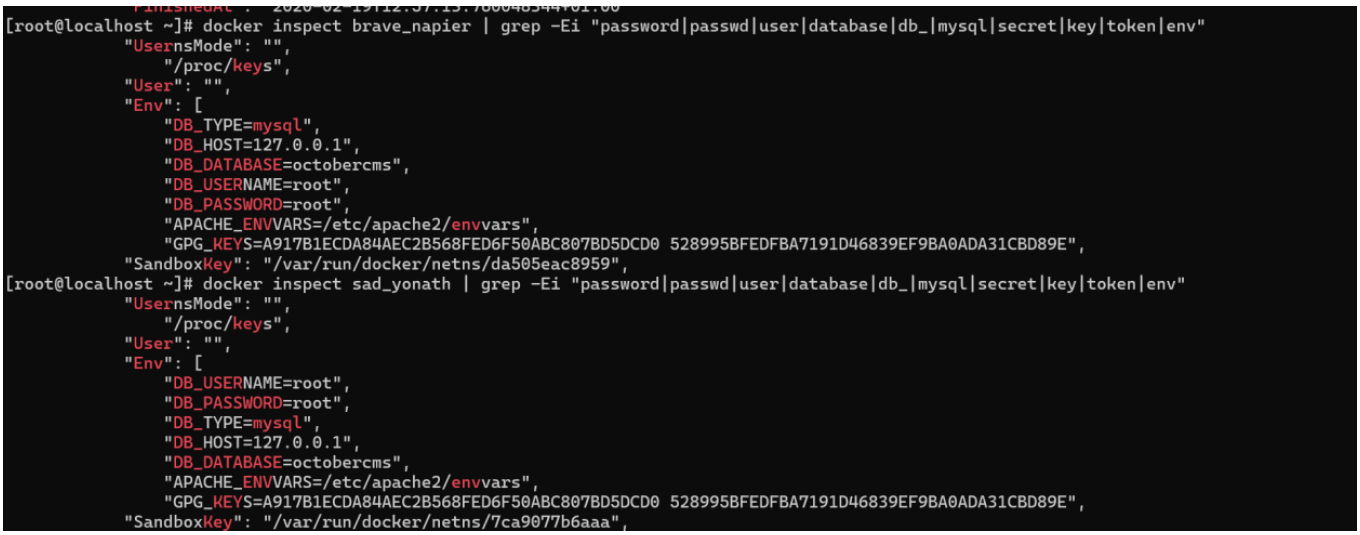
6. Finding 4 - Exposed Weak MySQL Root Credentials in Docker Metadata

Severity: Critical

OctoberCMS containers exposed database configuration directly in Docker environment variables. The vulnerable values included DB_USERNAME=root and DB_PASSWORD=root. This is a critical issue because anyone with Docker, host root, or Portainer access can read these values through docker inspect.

```
docker inspect brave_napier | grep -Ei "password|passwd|user|database|db_|mysql|secret|key|token|env"
docker inspect sad_yonath | grep -Ei "password|passwd|user|database|db_|mysql|secret|key|token|env"
```

Evidence - plaintext database credentials in container metadata



Why this is a vulnerability

Plaintext database credentials in Docker metadata can be retrieved by authorized or compromised Docker users. The risk is severe because the application used the MySQL root account with a weak password. If reused successfully, this could allow full database compromise.

Impact

- Read, modify, or delete CMS database content.
- Create new database users or modify privileges.
- Extract application data and sensitive information.
- Support further compromise of the CMS or Docker environment.

Remediation

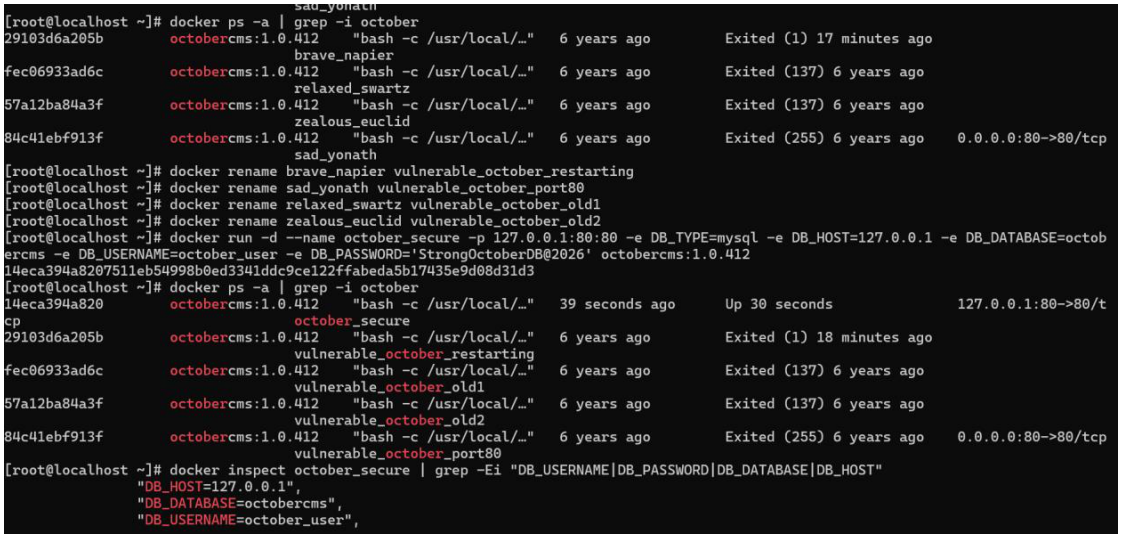
The vulnerable containers were stopped/renamed, and OctoberCMS was recreated with a dedicated database user and stronger password. The service was also bound to localhost only.

```
docker stop brave_napier sad_yonath relaxed_swartz zealous_euclid

docker run -d --name october_secure
  -p 127.0.0.1:80:80
  -e DB_TYPE=mysql
  -e DB_HOST=127.0.0.1
  -e DB_DATABASE=octobercms
  -e DB_USERNAME=october_user
  -e DB_PASSWORD='StrongOctoberDB@2026'
  octobercms:1.0.412

docker inspect october_secure | grep -Ei "DB_USERNAME|DB_PASSWORD|DB_DATABASE|DB_HOST"
```

Evidence - recreated OctoberCMS container with safer database values



7. Finding 5 - Sensitive Information Disclosure in OctoberCMS Logs

Severity: High

The OctoberCMS container logs disclosed backend routes, authentication activity, stack traces, service versions, MySQL errors, and sensitive user update values. Logs should not disclose internal application details or password-like values.

Evidence points:

- Backend routes were visible, including /backend/backend/auth/signin and /backend/backend/users/myaccount.
- Repeated HTTP 500 errors and traceback details were visible.
- The logs showed /usr/local/bin/bootstrap.py and a failed update_user function.
- A password-like value appeared inside a cms.update_user(...) call.
- MySQL version and security errors were visible, including mysqld 5.5.62-0+deb8u1.

```
docker logs vulnerable_october_port80 2>&1 | tail -50
docker logs vulnerable_october_port80 2>&1 | grep -Ei \
  "password|user|login|signin|error|traceback|exception|mysql|database|key|token"
docker logs vulnerable_october_restarting 2>&1 | tail -50
```

Why this is a vulnerability

Detailed logs help attackers understand the application structure and identify backend routes, software versions, operational errors, and sensitive values. If the exposed value is valid or reused, the risk can become critical.

Remediation

- Stop vulnerable containers that expose sensitive logs.
- Disable debug mode in OctoberCMS.
- Remove sensitive output from bootstrap or automation scripts.
- Avoid printing credentials, tokens, usernames, or password-like values in logs.
- Restrict Docker log access to trusted administrators only.
- Keep active services bound to localhost until tested securely.

```
docker stop vulnerable_october_port80 vulnerable_october_restarting
docker ps -a
docker exec -it october_secure /bin/bash
grep -Rni "debug" /var/www /var/www/html /app 2>/dev/null
```

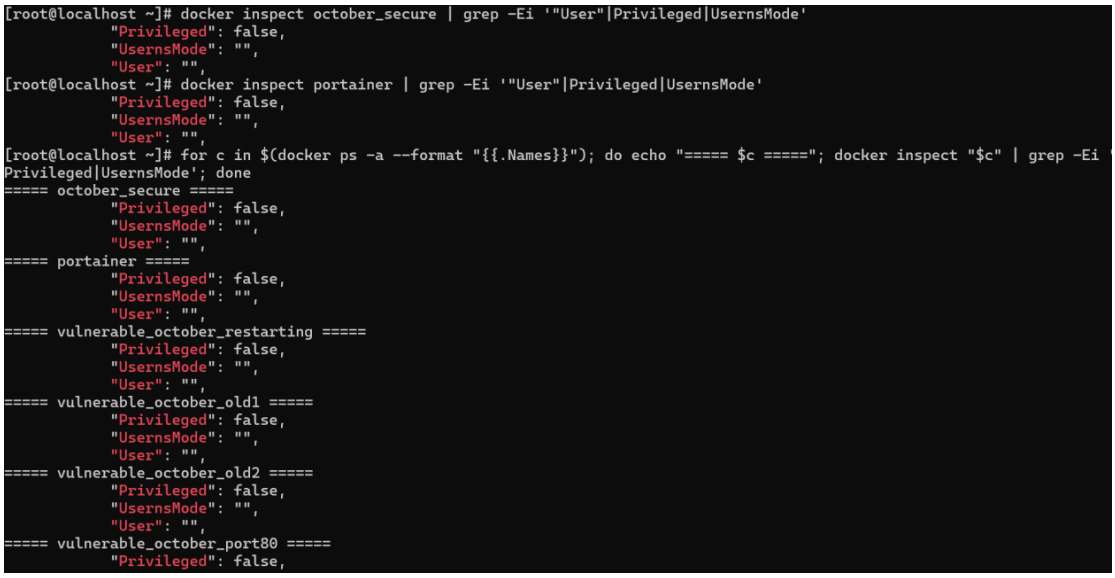
8. Finding 6 - Container Running as Root User

Severity: Medium / High

The Docker containers had an empty User field, meaning no non-root user was defined. When checked inside OctoberCMS, the container process was running as root.

```
docker inspect october_secure | grep -Ei '"User"|Privileged|UsernsMode'
docker inspect portainer | grep -Ei '"User"|Privileged|UsernsMode'
for c in $(docker ps -a --format "{{.Names}}"); do echo "===== $c ====="; docker inspect "$c" | grep -Ei \
  '"User"|Privileged|UsernsMode'; done
docker exec october_secure id
```

Evidence - containers did not define a non-root user



Why this is a vulnerability

If an attacker compromises the application, running as root inside the container increases the impact. The attacker may modify application files, access sensitive files, abuse mounted resources, or cause more damage than a limited user could.

Remediation

The OctoberCMS container was stopped, removed, and recreated with a non-root user using --user 1000:1000.

```
docker stop october_secure
docker rm october_secure
docker run -d --name october_secure
  --user 1000:1000
  -p 127.0.0.1:80:80
  -e DB_TYPE=mysql
  -e DB_HOST=127.0.0.1
  -e DB_DATABASE=octobercms
  -e DB_USERNAME=october_user
  -e DB_PASSWORD='StrongOctoberDB@2026'
  octobercms:1.0.412

docker exec october_secure id
```

Evidence - OctoberCMS verified as non-root after remediation

```
root@localhost ~]# docker exec october_secure id
id=0(root) gid=0(root) groups=0(root)
root@localhost ~]# docker ps
ONTAINER ID        IMAGE                 COMMAND                  CREATED             STATUS              PORTS
                   NAMES
4eca394a820        octobercms:1.0.412    "bash -c /usr/local/…"   26 minutes ago      Up 26 minutes       127.0.0.1:80->80/tcp
                   october_secure
779dc24dffc        portainer/portainer   "/portainer"             About an hour ago   Up About an hour    127.0.0.1:8000->8000/tcp,
27.0.0.1:9000->9000/tcp   portainer
root@localhost ~]# docker stop october_secure
ctober_secure
root@localhost ~]# docker rm october_secure
ctober_secure
root@localhost ~]# docker run -d --name october_secure --user 1000:1000 -p 127.0.0.1:80:80 -e DB_TYPE=mysql -e DB_HOST=127.0.0.1 -e
B_DATABASE=octobercms -e DB_USERNAME=october_user -e DB_PASSWORD='StrongOctoberDB@2026' octobercms:1.0.412
abaceb6ddb2f4e08e6801a846468925f8afd4f66e6f93a10abf19b31501a05e
root@localhost ~]# docker exec october_secure id
id=1000 gid=1000 groups=1000
```

9. Finding 7 - Outdated OctoberCMS Image and Legacy Runtime Components

Severity: High

The OctoberCMS environment used the old image tag octobercms:1.0.412. The logs also showed legacy runtime components such as Apache 2.4.10, PHP 7.1.3, and MySQL 5.5.62. These components may contain known vulnerabilities and may no longer receive security updates.

```
docker images | grep -i october
docker image inspect octobercms:1.0.412 | grep -Ei "Created|Version|Maintainer|RepoTags|DockerVersion"
docker exec october_secure apache2 -v
docker exec october_secure php -v
docker exec october_secure mysqld --version
```

Why this is a vulnerability

Outdated CMS and runtime components increase the risk of known public exploits, insecure dependencies, missing patches, and unsupported software behavior. The risk is higher if the application is exposed on a network-facing port.

Remediation

- Replace or rebuild the OctoberCMS image using supported versions.
- Upgrade PHP, Apache, and MySQL/MariaDB to supported versions.
- Test the updated image before exposing it.
- Remove old stopped containers after evidence is captured.
- Keep the service bound to 127.0.0.1 until patched and verified.

10. Final Risk Summary and Recommended Hardening

The most serious vulnerability is the exposed Portainer Docker management interface because it can provide administrative control over the Docker environment. The most dangerous supporting issue is exposed MySQL root credentials in Docker metadata because Portainer or Docker access could allow an attacker to retrieve database credentials and compromise OctoberCMS data.

- Keep Portainer bound to 127.0.0.1 and access it only through SSH tunneling or VPN.
- Upgrade Portainer to portainer/portainer-ce:latest after resolving DNS/CA certificate issues.
- Avoid using Docker environment variables for sensitive secrets where possible; use Docker secrets or a secure vault.
- Use dedicated least-privilege database accounts instead of MySQL root.
- Disable debug mode and remove sensitive values from logs.
- Run application containers as non-root users.
- Replace outdated OctoberCMS and runtime components with supported versions.
- Remove unused stopped containers after screenshots/evidence are no longer needed.
- Restrict Docker and root access to trusted administrators only.

Final status: major exposure issues were mitigated by localhost-only bindings and safer container configuration. The remaining high-priority tasks are software upgrades and long-term secrets management.